Cairo University - Faculty Of Engineering
Computer Engineering Department
Applied Data Science Project – Spring 2026

Flight Delay Classification Report

Team 5

Name	Sec	BN	Email
Ahmed Hamed Gaber Hamed	1	3	ahmed.hamed03@eng-st.cu.edu.eg
Akram Hany Karam Salam	1	13	akram.sallam03@eng-st.cu.edu.eg
Amir Anwar Bekhit Awd Kedis	1	14	amir.awd03@eng-st.cu.edu.eg
Somia Saad Ismail El-Shemy	1	26	somia.elshemy02@eng-st.cu.edu.eg

May 3, 2026

Contents

1 Abstract 2

2 Problem Statement 2

3 Target Variable Definition 3

4 Dataset Documentation 3

5 Merge Strategy 6

6 Engineered Features 6

7 Data Validation Report 7

8 Preprocessing 12

9 Transformation & Engineering 15

10 Exploratory Data Analysis (EDA) 16

11 Model Development 20

12 Experiment Tracking with MLflow 22

13 Testing 24

14 Results & Evaluation 24

15 Code Automation 27

16 Continuous Integration 28

17 Limitations & Future Work 28

18 Team Contributions 28

1 Abstract

Multi-class classification model to predict flight departure delay severity before pushback. Data sources: 2015 BTS domestic flight records (CSV) + Open-Meteo Historical Weather API. Four classes: **On Time**, **Minor Delay**, **Major Delay**, **Cancelled**. Final dataset: 116,381 rows × 55 columns. Validation found no full-row duplicates and only expected missing-value patterns.

2 Problem Statement

2.1 Business Problem

Flight delays cause operational disruptions, increased costs, and poor passenger experience. The goal is to **predict departure delay severity before pushback**, enabling ground operations to take proportional action in advance.

2.2 Stakeholders

Primary: Airline operations team and airport management.

Secondary: Passengers and customer service teams.

2.3 Why Classification

Delay severity is a discrete outcome, not a continuous one. Each class maps to a distinct operational response, making classification the best option to use. Multi-class over binary captures severity categories. Misclassification costs are asymmetric, missing a major delay is worse than over-alerting, which is handled through business-oriented evaluation metrics.

2.4 Decision-Making Impact

Operational Response by Predicted Class

Predicted Class	Label	Operational Action
`on_time`	On Time (≤15 min)	No action
`minor_delay`	Minor Delay (15–45 min)	Notify passengers, monitor connections
`major_delay`	Major Delay (>45 min)	Rebook connections, alert ground crew
`cancelled`	Cancelled	Trigger rebooking and compensation protocols

3 Target Variable Definition

The classification target is derived from the `DEPARTURE_DELAY` column (minutes) and the `CANCELLED` flag.

Class Distribution

Class	Label	Definition	Count	%
`on_time`	On Time	`DEPARTURE_DELAY` $\leq$ 15 min (FAA standard)	94,345	81.1%
`minor_delay`	Minor Delay	15 $<$ `DEPARTURE_DELAY` $\leq$ 45 min	11,433	9.8%
`major_delay`	Major Delay	`DEPARTURE_DELAY` $>$ 45 min	8,833	7.6%
`cancelled`	Cancelled	`CANCELLED == 1`	1,770	1.5%

Why `DEPARTURE_DELAY`: The goal is pre-departure action, departure delay is the directly actionable outcome.

Class thresholds: The 15-minute threshold follows the FAA on-time standard. The 45-minute split is where airlines typically begin rebooking connecting passengers.

Cancelled flights: Retained as a distinct fourth class. They have no `DEPARTURE_DELAY` but cancellation is itself a severe outcome. Final handling deferred to the preprocessing phase.

Class imbalance: `on_time` dominates at 81.1%. Will be addressed using **SMOTE and other techniques while training**.

4 Dataset Documentation

4.1 Source 1, CSV (BTS / Kaggle)

Dataset Overview

Field	Details
Name	2015 Flight Delays and Cancellations
Provider	US Department of Transportation, Bureau of Transportation Statistics (BTS)
URL	kaggle.com/datasets/usdot/flight-delays
Citation	U.S. Dept. of Transportation, BTS. (2015). *Airline On-Time Performance Data*. Kaggle.
Acquisition	Direct CSV download (flat file)
Raw Size	~5,819,079 flight records $\times$ 31 features
Sampled Size	116,381 rows (2% stratified random sample by month, `random_state=42`)
Coverage	All domestic US flights, calendar year 2015
License	Public domain (US government data)

Key Features:

BTS Feature Descriptions

Feature	Type	Description
YEAR	int	Year of flight (2015)
MONTH	int	Month (1–12)
DAY	int	Day of month
DAY_OF_WEEK	int	1=Monday … 7=Sunday
AIRLINE	str	IATA airline carrier code (e.g., AA, DL, UA)
FLIGHT_NUMBER	int	Airline-assigned flight number
TAIL_NUMBER	str	Aircraft tail registration number
ORIGIN_AIRPORT	str	IATA code of departure airport
DESTINATION_AIRPORT	str	IATA code of arrival airport
SCHEDULED_DEPARTURE	int	Scheduled departure time (HHMM)
DEPARTURE_TIME	float	Actual departure time, *post-departure, excluded*
DEPARTURE_DELAY	float	Departure delay in minutes, *post-departure, excluded*
TAXI_OUT	float	Taxi-out time in minutes, *post-departure, excluded*
WHEELS_OFF	float	Wheels-off time (HHMM), *post-departure, excluded*
SCHEDULED_TIME	float	Scheduled flight duration (minutes)
ELAPSED_TIME	float	Actual elapsed time, *post-departure, excluded*
AIR_TIME	float	Airborne time, *post-departure, excluded*
DISTANCE	int	Flight distance (miles)
WHEELS_ON	float	Wheels-on time, *post-departure, excluded*
TAXI_IN	float	Taxi-in time, *post-departure, excluded*
SCHEDULED_ARRIVAL	int	Scheduled arrival time (HHMM)
ARRIVAL_TIME	float	Actual arrival time, *post-departure, excluded*
ARRIVAL_DELAY	float	Arrival delay in minutes, used to build target label only
DIVERTED	int	1 if flight was diverted
CANCELLED	int	1 if flight was cancelled
CANCELLATION_REASON	str	A=Carrier, B=Weather, C=NAS, D=Security
AIR_SYSTEM_DELAY	float	Delay from NAS, *post-departure, excluded*
SECURITY_DELAY	float	Delay from security, *post-departure, excluded*
AIRLINE_DELAY	float	Delay from carrier, *post-departure, excluded*
LATE_AIRCRAFT_DELAY	float	Delay from late aircraft, *post-departure, excluded*
WEATHER_DELAY	float	Delay from weather, *post-departure, excluded*

4.2 Source 2, REST API (Open-Meteo Historical Weather)

API Overview

Field	Details
Name	Open-Meteo Historical Weather API
Provider	Open-Meteo
URL	open-meteo.com/en/docs/historical-weather-api
Cost	Free, no API key required
Acquisition	HTTP GET requests, one call per unique airport per year, cached as JSON in `data/raw/weather_cache/{IATA}_{YEAR}.json`
Query Parameters	`latitude`, `longitude`, `start_date=2015-01-01`, `end_date=2015-12-31`, `hourly` variables, `timezone=America/New_York`

Weather Variables Retrieved (per airport, per hour):

Weather Feature Descriptions

Feature (origin)	Feature (dest)	Unit	Description
`temperature_c`	`dest_temperature_c`	°C	Air temperature at 2 m
`precipitation_mm`	`dest_precipitation_mm`	mm	Total precipitation
`rain_mm`	`dest_rain_mm`	mm	Rain component
`snowfall_cm`	`dest_snowfall_cm`	cm	Snowfall
`wind_speed_kmh`	`dest_wind_speed_kmh`	km/h	Wind speed at 10 m
`wind_direction_deg`	`dest_wind_direction_deg`	deg	Wind direction
`wind_gusts_kmh`	`dest_wind_gusts_kmh`	km/h	Wind gusts
`cloud_cover_pct`	`dest_cloud_cover_pct`	%	Cloud cover
`weather_code`	`dest_weather_code`	WMO	Precipitation/fog/storm type
`relative_humidity_pct`	`dest_relative_humidity_pct`	%	Relative humidity
`pressure_msl_hpa`	`dest_pressure_msl_hpa`	hPa	Sea-level pressure

Why destination weather? *Destination weather at scheduled arrival time is knowable before departure (weather forecasts) and directly influences whether the plane can land. Including it improves the model's ability to predict arrival delays caused by destination-side conditions.*

5 Merge Strategy

Type: Horizontal merge, weather columns are appended to each flight row.

Join Keys

Join	Key	Weather Columns Added
Origin weather	`(ORIGIN_AIRPORT, MONTH, DAY, departure_hour)`	11 cols (`temperature_c`, ...)
Destination weather	`(DESTINATION_AIRPORT, MONTH, arrival_day, scheduled_arrival_hour)`	11 cols (`dest_temperature_c`, ...)

Timezone handling: The Open-Meteo API returns timestamps in `America/New_York`. Flight schedules are in each airport's local time. `timezonefinder` resolves each airport's IANA timezone from its coordinates, `zoneinfo` converts local departure/arrival hours to Eastern before building the lookup key, correctly handling DST for all 2015 dates.

Overnight flights: Arrival day is computed by adding `SCHEDULED_TIME` (minutes) to departure hour, detecting day rollovers where the flight arrives the next calendar day.

Row Counts Before/After Merging

Stage	Row Count
Raw BTS flights CSV	~5,819,079
After 2% stratified sample (by month)	116,381
After weather join (no rows dropped)	116,381
Rows with missing origin weather (numeric airport codes)	9,837 (8.45%)
Rows with missing destination weather (numeric airport codes)	9,823 (8.44%)
Final dataset	**116,381 × 55 cols**

The join does not drop rows, flights with no valid airport code get `NaN` weather values. These ~9,837 rows will be dropped in the cleaning phase as weather is a core feature set.

6 Engineered Features

Features derived during acquisition (present in `flights_weather.csv`):

Acquired Engineered Features

Feature	Derivation	Notes
`departure_hour`	`SCHEDULED_DEPARTURE // 100`	Hour of scheduled departure (0–23)
`scheduled_arrival_hour`	`SCHEDULED_ARRIVAL // 100`	Hour of scheduled arrival (0–23)
`dest_*` weather columns	Open-Meteo API at destination	11 destination weather features

Additional features to be engineered during the preprocessing phase:

- `is_weekend`, `is_holiday`
- `route` (ORIGIN + DESTINATION)

- Historical delay rates per airline/route
- `busy_bad_weather_score`

7 Data Validation Report

Validation was performed via `src/validation/validate.py` covering 12 checks. Results are reproduced from `notebooks/validation_report.ipynb`.

Dataset: *`data/processed/flights_weather.csv`, 116,381 rows × 55 columns*

7.1 Summary

Validation Checks Summary

Check	Result
Schema	All 55 columns present, all types are correct
Completeness	Weather cols 8.45% null (numeric airports, expected), delay breakdown 81.75% null (expected, only set when delayed), `TAIL_NUMBER` 0.22% null
Duplicates	0 full-row duplicates, 864 flight-level duplicates (codeshares, expected)
Value Ranges	All numeric values within expected bounds
Consistency	97 cancelled flights have a `DEPARTURE_TIME` (taxied before cancellation)
Outliers	Detected via IQR, delay and distance columns show expected right-skewed extremes
Referential Integrity	9,723 rows use numeric FAA IDs instead of IATA codes, no coordinate match
Format Validation	All IATA codes, HHMM times, and YEAR values conform to expected formats
Temporal Validity	All (MONTH, DAY) combinations are valid calendar dates
Cardinality	14 unique airlines, ∼322 unique origin and destination airports
Statistics	Descriptive statistics within expected bounds for all numeric columns
Target Readiness	340 non-cancelled rows unlabellable (no `ARRIVAL_DELAY`), 4-class distribution documented

7.2 Completeness, Missing Values

Missing Value Analysis

Column	Missing Count	Missing %	Explanation
CANCELLATION_REASON	114,611	98.48%	Cancelled flights only, expected
LATE_AIRCRAFT_DELAY	95,143	81.75%	Delayed flights only, expected
AIR_SYSTEM_DELAY	95,143	81.75%	Same, expected
SECURITY_DELAY	95,143	81.75%	Same, expected
WEATHER_DELAY	95,143	81.75%	Same, expected
AIRLINE_DELAY	95,143	81.75%	Same, expected
Origin weather cols (11)	9,837	8.45%	Numeric airport codes, no IATA match
Dest weather cols (11)	9,823	8.44%	Same (slightly different set)
ARRIVAL_DELAY	2,110	1.81%	Cancelled + 340 diverted, expected
AIR_TIME / ELAPSED_TIME	2,110	1.81%	Cancelled/diverted, expected
TAXI_IN / WHEELS_ON / ARRIVAL_TIME	1,829	1.57%	Flights that did not arrive
WHEELS_OFF / TAXI_OUT	1,748	1.50%	Flights that did not depart
DEPARTURE_TIME / DEPARTURE_DELAY	1,673	1.44%	Cancelled flights
TAIL_NUMBER	258	0.22%	Missing tail registration

7.3 Duplicates

Duplicate Check

Check	Count	Notes
Full-row duplicates	0	No exact row duplicates
Flight-level duplicates (AIRLINE + FLIGHT_NUMBER + MONTH + DAY)	864	Codeshare flights, multiple airlines operating same physical flight, expected

7.4 Value Ranges

Numeric Column Range Checks

Column	Valid Range	Violations
MONTH	1–12	0
DAY	1–31	0
DAY_OF_WEEK	1–7	0
SCHEDULED_DEPARTURE	0–2359	0
DISTANCE	> 0	0
SCHEDULED_TIME	> 0	0

~9,837 rows have numeric values in ORIGIN_AIRPORT (e.g., 14747) that are FAA numeric station IDs rather than IATA codes. These are not range violations, they are a data quality issue documented under Referential Integrity.

7.5 Consistency Checks

Consistency Results

Check	Rows Affected	Description
Canceled flights with DEPARTURE_TIME	97	Flights that was canceled while on ground before they took off (due to any issues)

7.6 Outlier Detection (IQR Method, 1.5× Fence)

Outliers were detected using the IQR method across key numeric columns. Extreme values in delay-related columns are expected (heavy weather events, ground stops) and will be handled during the cleaning phase by binning rather than dropping.

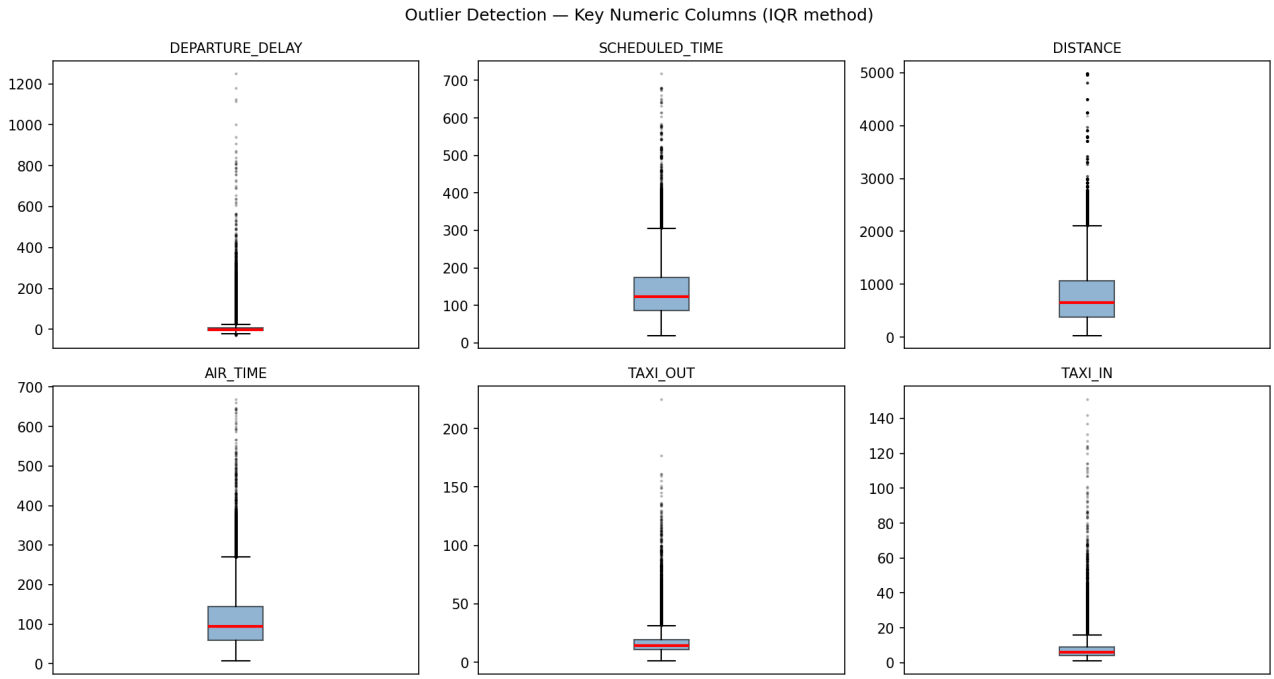


Figure 1: Outliers using IQR method

7.7 Referential Integrity

Referential Integrity Checks

Check	Unmatched Count	Unmatched Values (sample)
`AIRLINE` codes vs `airlines.csv`	0	All match
IATA airport codes vs `airports.csv`	9,723	Numeric FAA IDs (e.g., 14747, 13830), no IATA equivalent

The 9,723 rows with numeric airport codes (∼8.35% of the dataset) have no coordinates and therefore no weather data. **Decision:** drop in the cleaning phase, weather is a core feature set and imputation would be meaningless.

7.8 Format Validation

Format Checks

Check	Violations
IATA codes (3 uppercase letters)	0 (among non-numeric codes)
HHMM times (minutes 00–59)	0
`YEAR` = 2015	0
`CANCELLATION_REASON` in {A, B, C, D}	0

7.9 Temporal Validity

All (MONTH, DAY) combinations are valid calendar dates for 2015. No invalid dates detected.

7.10 Cardinality

Cardinality Summary

Column	Unique Values	Top Values
`AIRLINE`	14	WN, DL, AA, UA, OO
`ORIGIN_AIRPORT`	∼322	ATL, ORD, DFW, LAX, DEN
`DESTINATION_AIRPORT`	∼322	ATL, ORD, DFW, LAX, DEN
`DAY_OF_WEEK`	7	,
`CANCELLATION_REASON`	4	B (Weather), A (Carrier), C (NAS), D (Security)

7.11 Target Variable Readiness

Target Readiness Metrics

Metric	Value
Total rows	116,381
Cancelled flights	1,770
`DEPARTURE_DELAY` nulls (non-cancelled)	0
Unlabellable rows	0
Labellable rows	116,381

Class distribution (target: `DEPARTURE_DELAY`):

Final Class Distribution

Class	Label	Count	%
`on_time`	On Time (≤15 min)	94,345	81.1%
`minor_delay`	Minor Delay (15–45 min)	11,433	9.8%
`major_delay`	Major Delay (>45 min)	8,833	7.6%
`cancelled`	Cancelled	1,770	1.5%

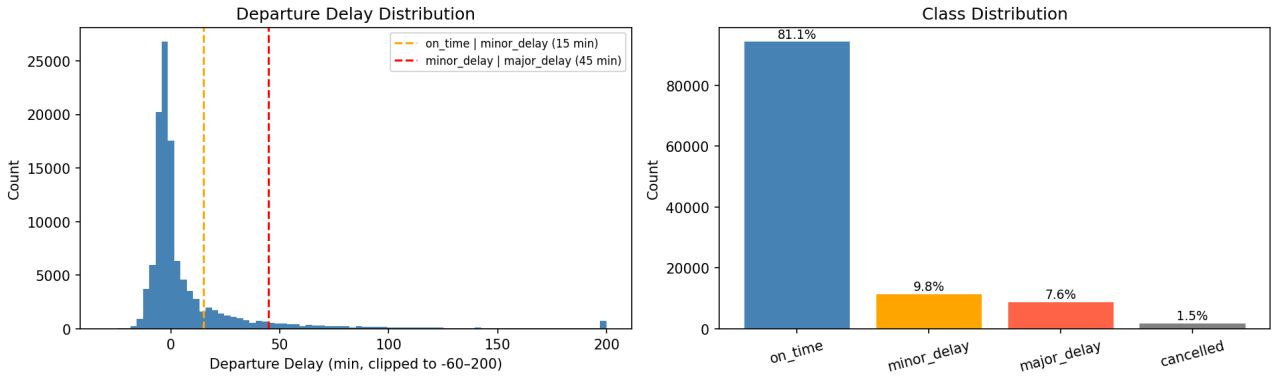


Figure 2: Departure Delay Classes

7.12 Known Issues and Planned Resolutions

Known Issues

Issue	Count	Severity	Planned Resolution
Rows with numeric airport codes (no weather)	9,723	Medium	Drop in cleaning phase
Cancelled flights with `DEPARTURE_TIME`	97	Low	Document as data quirk, keep
Missing `TAIL_NUMBER`	258	Low	Drop column, not a model feature
Delay breakdown cols (post-departure leakage)	5 cols	High	Drop columns in cleaning phase

8 Preprocessing

The preprocessing pipeline consists of several sequential transformations applied to the raw merged flight and weather dataset.

8.1 Removal of Data Leakage and Identifiers

To prevent data leakage, all post-departure variables that would not be available at the time of prediction were removed. The exact features that were excluded during this stage and the rationale behind their removal are outlined below.

Dropped Leakage and Identifier Columns

Dropped Column	Reason for Removal
FLIGHT_NUMBER	Not relevant.
TAIL_NUMBER	Not relevant.
DEPARTURE_TIME	Target leakage.
ARRIVAL_TIME	Target leakage.
WHEELS_OFF	Target leakage.
WHEELS_ON	Target leakage.
ELAPSED_TIME	Target leakage.
AIR_TIME	Target leakage.
TAXI_OUT	Target leakage.
TAXI_IN	Target leakage.
ARRIVAL_DELAY	Not a pre-departure feature.
AIR_SYSTEM_DELAY	Delay breakdown (target leakage).
SECURITY_DELAY	Delay breakdown (target leakage).
AIRLINE_DELAY	Delay breakdown (target leakage).
LATE_AIRCRAFT_DELAY	Delay breakdown (target leakage).
WEATHER_DELAY	Delay breakdown (target leakage).

8.2 Target Variable Creation

The primary objective of the classification model is to predict the severity of flight delays. A new target variable, DELAY_CATEGORY, was engineered based on the continuous DEPARTURE_DELAY and the CANCELLED binary indicator:

- **On Time:** Flights departing up to 15 minutes after the scheduled time (adhering to the standard FAA definition of on-time performance).
- **Minor Delay:** Flights delayed between 16 and 45 minutes.
- **Major Delay:** Flights delayed for more than 45 minutes.
- **Cancelled:** Flights explicitly marked as cancelled in the original dataset.

Any rows lacking sufficient data to be categorized into one of these four discrete classes were subsequently dropped.

8.3 Data Cleaning and Filtering

Several data quality checks and filtering mechanisms were applied to ensure the integrity of the feature space:

- **Invalid Airport Codes:** Records containing numeric IATA airport codes (e.g., 12345) were discarded, as they indicate corrupted or incorrectly mapped origin or destination data.
- **Null Handling:** Any residual rows containing null values in mandatory feature columns were removed to ensure dataset completeness. Exceptions were made for conditionally null columns such as CANCELLATION_REASON, which are logically empty for completed flights.

8.4 Outlier Treatment

To improve the robustness of downstream machine learning models against extreme, unrepresentative values, statistical outlier capping was performed. Continuous numeric features-such as DISTANCE, SCHEDULED_TIME, and weather-related metrics (temperature_c, wind_speed_kmh)-were capped at their 1st and 99th percentiles.

8.5 Post-Cleaning Validation

As a final step, the preprocessed dataset was automatically passed through a secondary validation suite. This ensured that no residual nulls existed in the feature space and that all values adhered to expected logical ranges before the data was serialized and passed to the feature engineering phase.

9 Transformation & Engineering

We performed an extensive feature engineering pipeline. We enriched the dataset by engineering temporal variables, interaction keys, operational congestion ratios, and smoothed historical target encodings. To ensure numerical stability for gradient-descent, a `StandardScaler` was subsequently applied within the modeling pipeline.

The complete list of engineered features is detailed in the table below.

Engineered Features

Feature	Type	Description
`is_weekend`	int	1 if flight is on Saturday or Sunday
`is_holiday`	int	1 if flight falls on a US Federal Holiday
`scheduled_departure_hour_sin` `scheduled_departure_hour_cos`	float	Cyclical sine/cosine encoding of scheduled departure hour
`scheduled_arrival_hour_sin` `scheduled_arrival_hour_cos`	float	Cyclical sine/cosine encoding of scheduled arrival hour
`month_sin` `month_cos`	float	Cyclical sine/cosine encoding of flight month
`departure_hour_bucket`	str	Discretized bucket (e.g., morning, midday)
`departure_hour_bucket_code`	int	Numeric label of departure_hour_bucket
`is_morning_peak_bank`	int	1 if departure hour is in 06:00 and 09:00
`is_evening_peak_bank`	int	1 if departure hour is in 16:00 and 19:00
`ROUTE`	str	Concatenation of Origin and Destination
`AIRLINE_DEPARTURE_BANK`	str	Airline concatenated with departure bucket
`ORIGIN_AIRPORT_DEPARTURE_BANK`	str	Origin concatenated with departure bucket
`ORIGIN_AIRPORT_DAY_OF_WEEK`	str	Origin concatenated with day of week
`AIRLINE_ROUTE`	str	Airline concatenated with route
`ROUTE_DEPARTURE_BANK`	str	Route concatenated with departure bucket
`ROUTE_DAY_OF_WEEK`	str	Route concatenated with day of week
`*_count` *(e.g., origin_hourly_departure_count)*	int	Hourly or bank departure/arrival volume counts for specific locations
`*_ratio` *(e.g., origin_congestion_ratio)*	float	Ratio of current traffic volume to the historical mean volume
`*_weather_intensity`	float	Composite weather severity score
`*_weather_score` *(e.g., route_congestion_weather_score)*	float	Interaction term: local congestion ratio × local weather intensity
`*_freq`	float	Frequency encoding to represent the volume of specific routing columns
`*_historical_*_rate`	float	Smoothed, leave-one-out historical probability of a specific delay class for a specific entity

9.1 Feature selection tried

We also tried a small **filter-based feature-selection step** during training. The method was **mutual information**: each feature was scored against the training labels, and only columns above a minimum threshold were kept. In code, this was implemented as an optional `mutual_info` mode with a default `min_mutual_info` threshold of `0.001`. If no column passed the threshold, the pipeline kept the single best varying feature as a fallback.

We kept this as an experiment rather than making it part of the final default pipeline. In the ablation runs, the mutual-information filter gave only negligible changes and did not consistently improve test performance. For example, HistGradientBoosting stayed slightly better without it (balanced accuracy about **0.378** without selection vs **0.373** with mutual information), while balanced logistic regression changed only marginally (about **0.396** without selection vs **0.397** with it). Because the gain was small and inconsistent, the final system relied more on **feature engineering plus model-specific feature adaptation** than on aggressive column pruning.

10 Exploratory Data Analysis (EDA)

The EDA was done on the cleaned modeling dataset with **106,444 rows** and **38 columns**. The goal was not only to describe the data, but to answer the questions that matter before modeling: how severe the class imbalance is, which features appear useful, where delays concentrate, and which columns look redundant or need special treatment.

10.1 Target Distribution

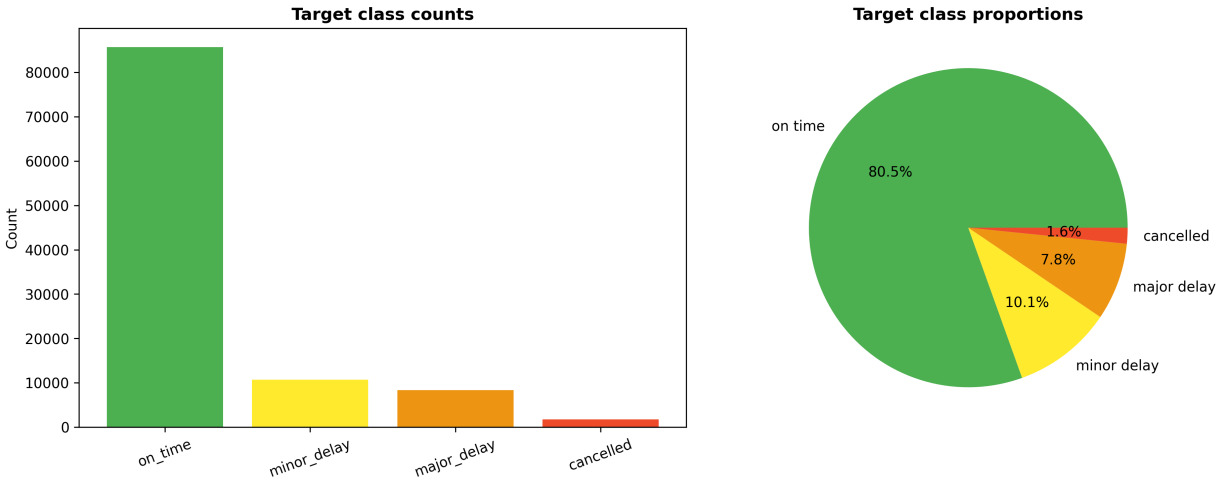


Figure 3: Target distribution in the cleaned dataset, shown as both counts and proportions.

The first thing the eda makes clear is that the task is strongly imbalanced. About **80.5%** of flights are `on_time`, while `minor_delay`, `major_delay`, and `cancelled` make up the much smaller remainder. That matters immediately for modeling. A classifier can get a deceptively high accuracy score (foreshadowing) just by staying close to the majority class, which is exactly why balanced metrics and business-aware costs were needed later.

10.2 Flight Activity Context

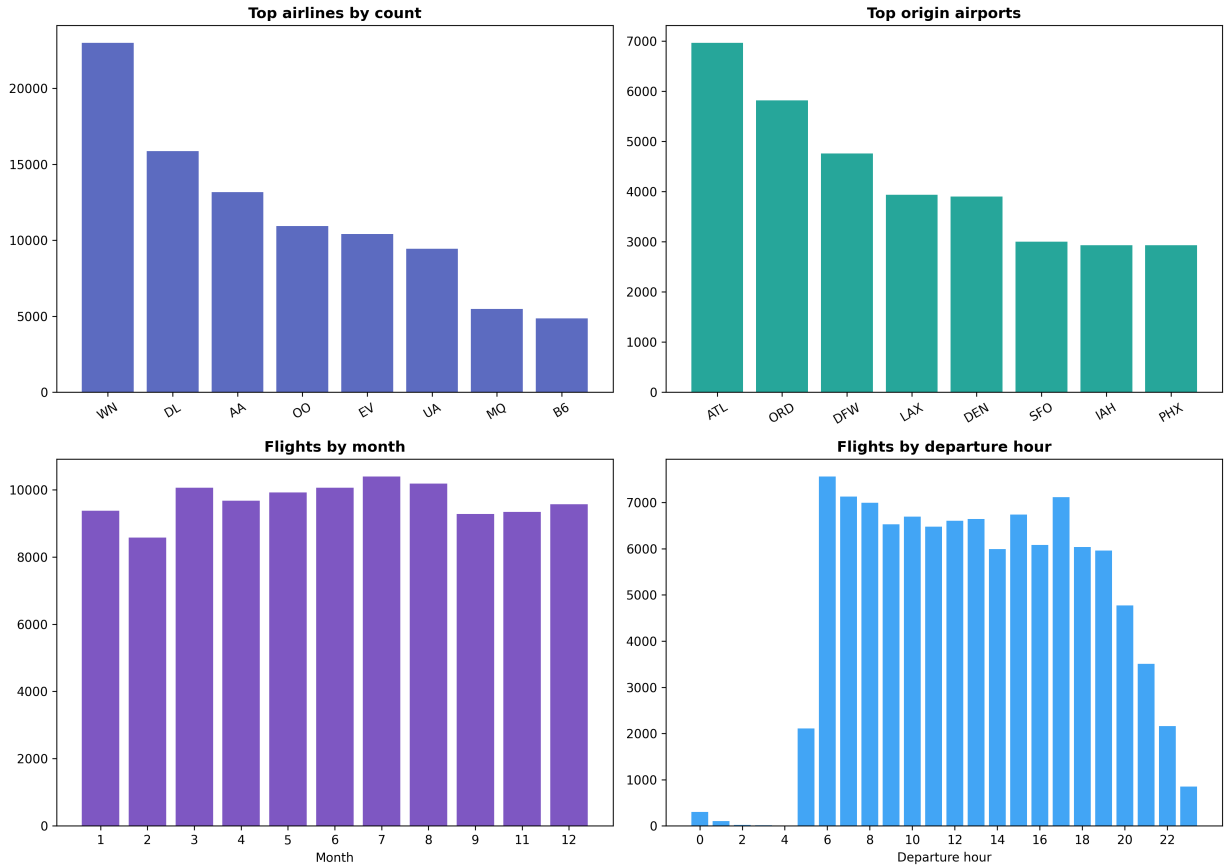


Figure 4: Context plots for the cleaned dataset: dominant airlines, busiest origin airports, monthly coverage, and departure-hour frequency.

This figure gives the basic operating context of the dataset. Southwest (`WN`) is the largest airline by volume in the sample, while airports such as `ATL`, `ORD`, `DFW`, and `DEN` dominate the origin counts.

10.3 Delay Mix by Airline and Month

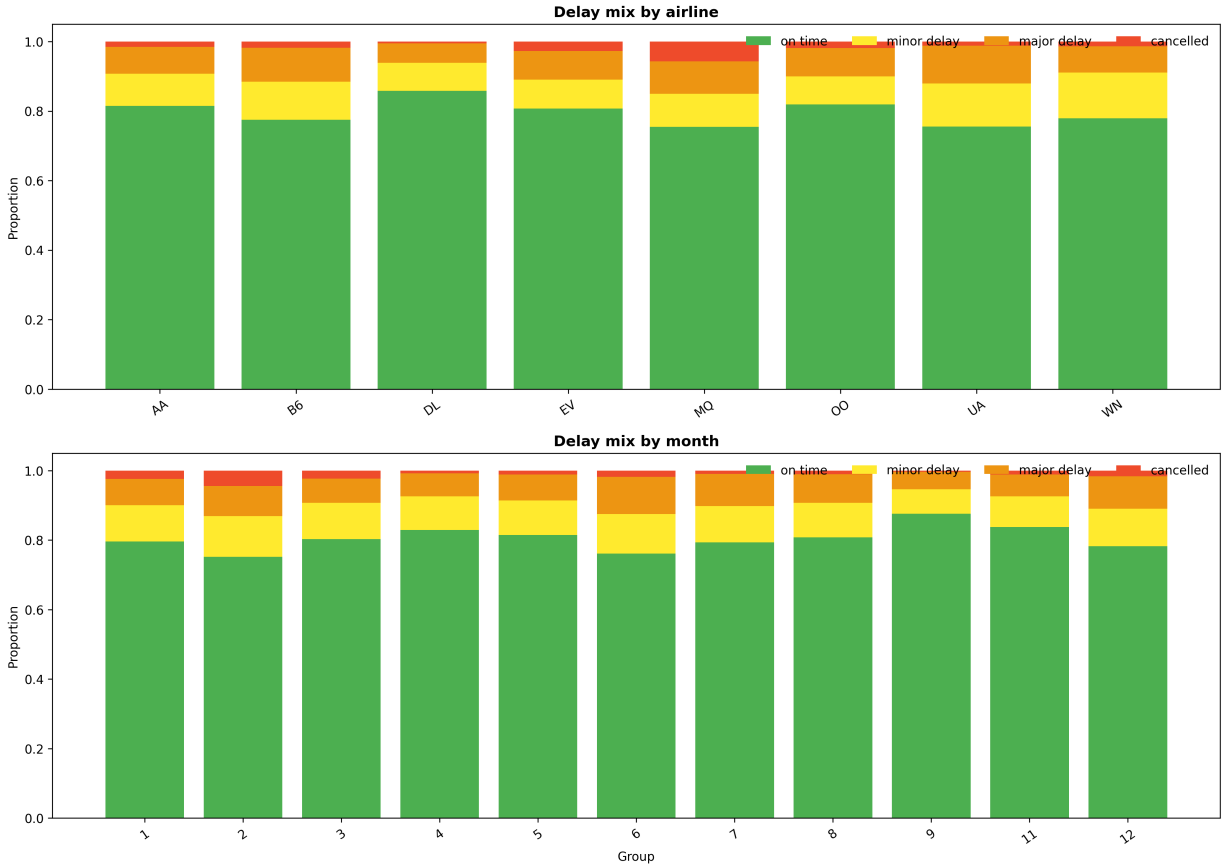


Figure 5: Normalized delay-category mix by airline and by month. Raw counts were converted to within-group proportions so minority classes remain visible.

Raw counts hide too much because `on_time` dominates the dataset, so the eda switched to within-group proportions. That view is much more useful. Some airlines, especially `NK` and `F9`, carry a visibly larger share of `major_delay` and `cancelled` flights than the stronger carriers. The month view shows a similar pattern: winter months such as **January, February, and December** are worse, and **June and July** also show elevated disruption. These plots justified keeping airline and calendar features in the modeling pipeline.

10.4 Delay Rate by Departure Hour

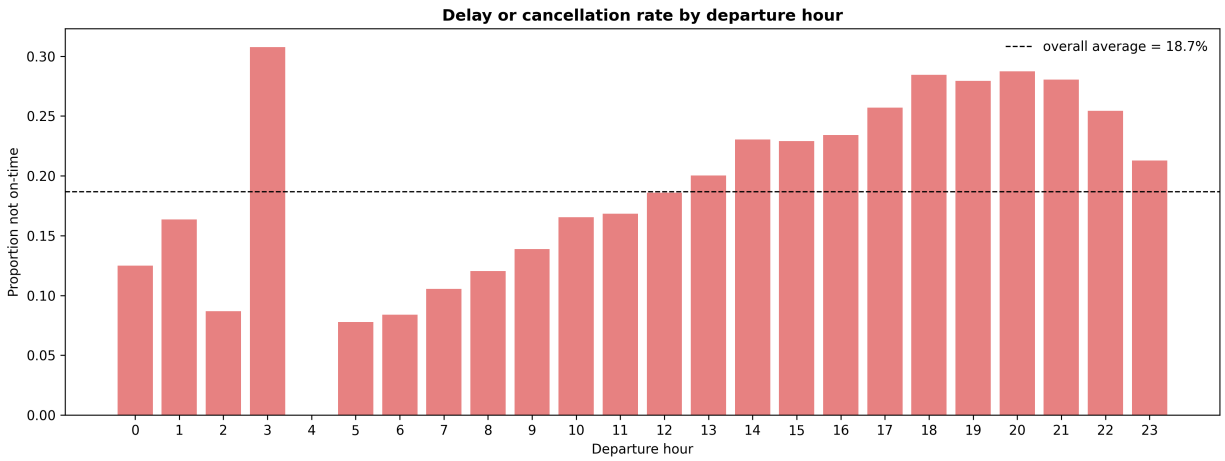


Figure 6: Share of flights that are not on time by scheduled departure hour, with the overall average shown as a dashed reference line.

Departure hour turned out to be one of the clearest temporal signals in the notebook. Flights leaving between roughly **14:00 and 22:00** have the highest not-on-time rate, while early-morning departures, especially around **05:00 to 07:00**, are the most reliable. That fits the usual delay-propagation story: once the day gets busy, late aircraft, gate pressure, and congestion start to compound. This is one of the main reasons we kept `departure_hour` and later engineered time-based features around it.

10.5 Numerical Features vs the Target

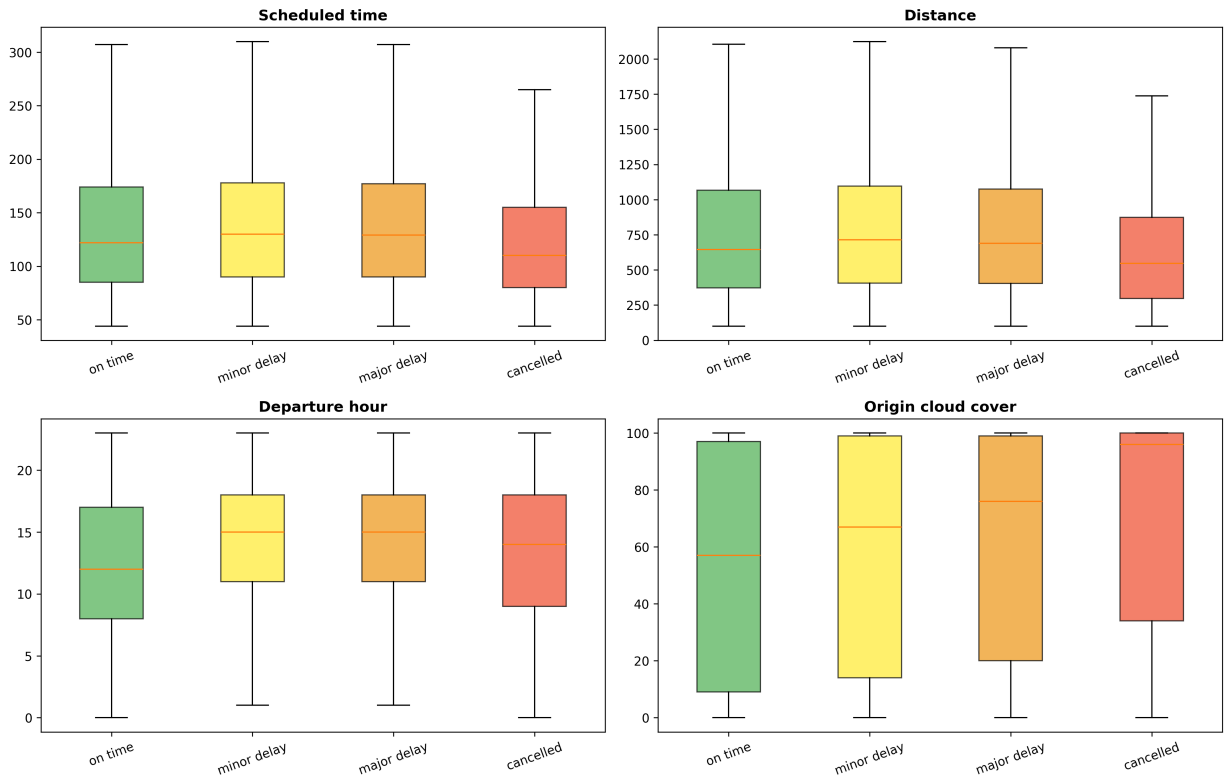


Figure 7: Selected numerical features plotted against the delay class. The plots focus on scheduled time, distance, departure hour, and cloud cover.

The notebook's box-plot analysis showed a few useful patterns, even though none of them by themselves cleanly separate the classes. Flights with `major_delay` or `cancelled` labels tend to have slightly **longer scheduled times and distances**. Delayed and cancelled flights also tend to leave **later in the day**, which supports the departure-hour result above. Weather variables were mixed: temperature alone did not separate the classes much, but **cloud cover** and some wind-related variables were slightly worse for delayed and cancelled flights.

10.6 Correlation Analysis and Redundancy

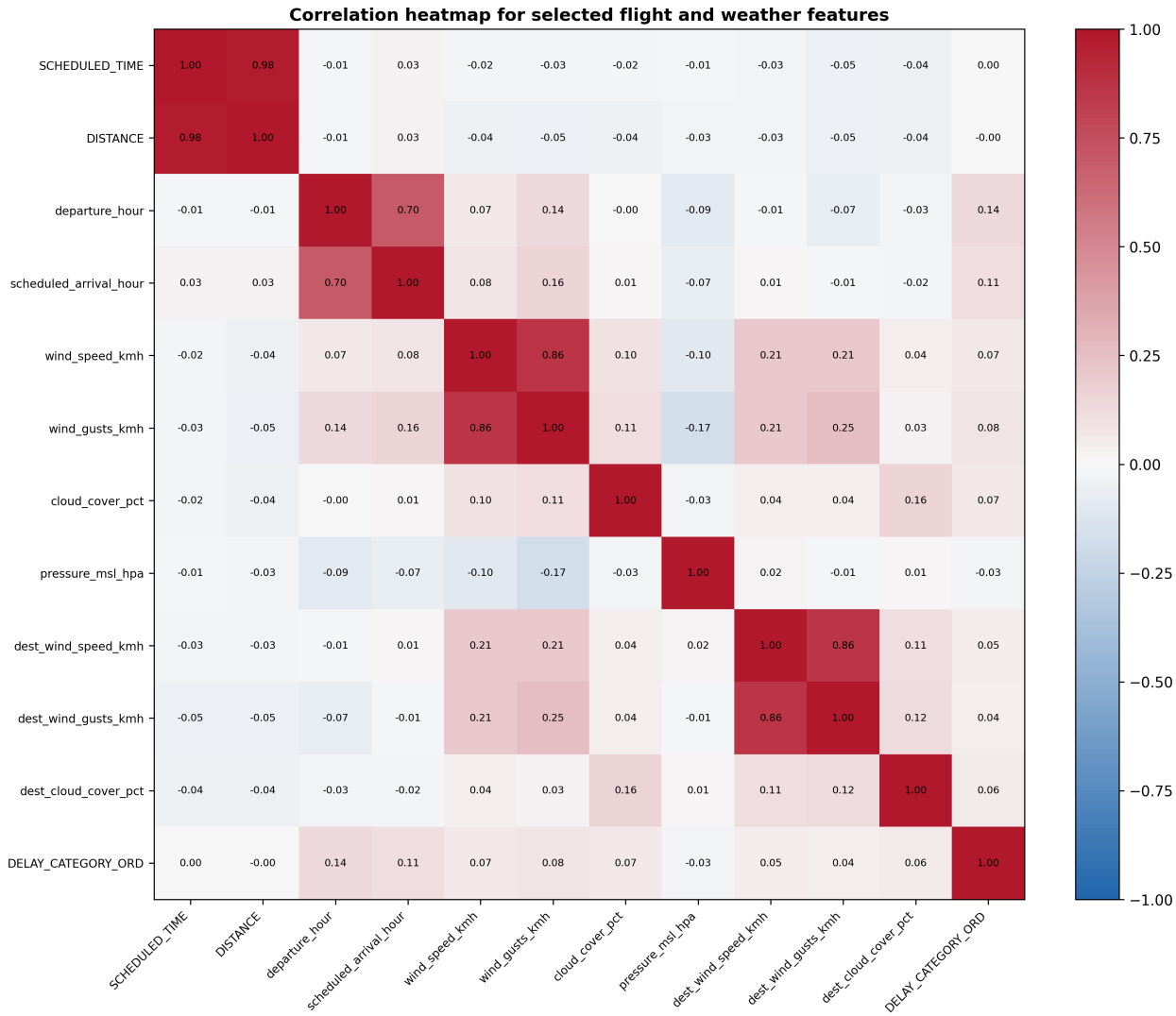


Figure 8: Correlation heatmap for selected flight and weather features, including an ordinal encoding of the target for rough linear comparison.

The target correlation stayed weak across all individual features **the problem is not strongly linear**, and feature interactions matter more than any single raw column.

10.7 EDA Takeaways That Affected Modeling

The EDA directly shaped the modeling choices used later in the project:

- Severe class imbalance made plain accuracy unreliable and justified class weighting, resampling experiments, and balanced evaluation metrics.
- Airline, month, and departure-hour patterns gave strong support for keeping temporal and categorical context features.
- Weak linear target correlations suggested that non-linear models would likely outperform a purely linear baseline once the feature representation improved.

11 Model Development

This stage focused on a simple question: how well can we predict delay severity or cancelation before departure using the same cleaned inputs for every model? All final models were tested on the same untouched test split,

and compared with the same primary metric, **balanced accuracy**. That choice matters here because the dataset is heavily imbalanced and plain accuracy mostly rewards models that keep predicting `on_time`.

We could have chosen easier metrics to achieve but they would have hurt the business more than benefited it.

11.1 Final Benchmark Ladder

Candidate Models Used in the Locked Final Comparison

Model	Role	Why It Was Included
Majority baseline	Reference model	Sanity check class imbalance
Logistic regression (unbalanced)	Linear test	Shows what happens when the class distribution is left untreated
Logistic regression (balanced)	Linear reference model	First logical choice for our classification task
Random forest	Non-linear tree baseline	Captures non-linear interactions without feature scaling
Extra trees	Tree ensemble variant	Checks whether heavier tree randomization helps on this feature set
HistGradientBoosting	Main boosted-tree model	It's best used when we have dense tabular features and better minority-class recall
Hierarchical HistGradientBoosting	Two-stage extension	Separates `on_time` vs delayed first, then splits delayed flights by severity the idea that we were facing an issue on saperating on time from other classes)
XGBoost (balanced)	Additional boosted-tree test	Tests whether XGBoost beats the other tree models.

We also logged an exploratory balanced MLP (DL model) run during the experiments. It reached only **0.363 balanced accuracy** on the earlier runs, so we dropped.

11.2 Simplified Modeling Pipeline Snapshot

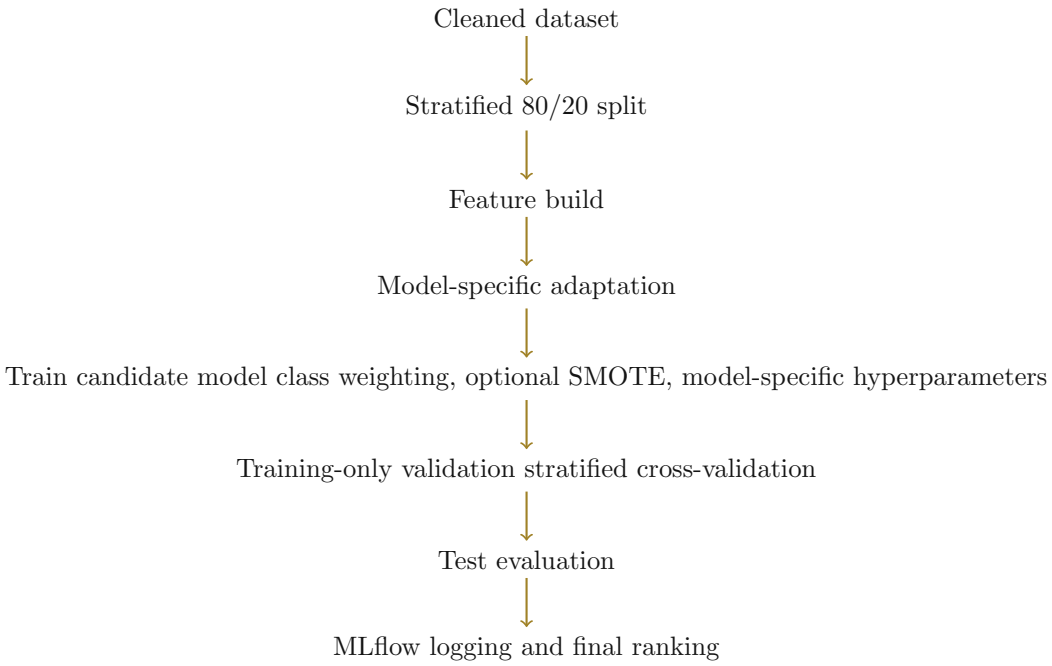


Figure 9: Simplified model-development pipeline used for the final benchmark ladder.

11.3 Training, Validation, and Tuning Strategy

- **Train/Testing split:** the cleaned dataset was split with a class-stratified 80/20 way. The winning final run used **85,154 training rows**, **21,290 holdout rows**, and **61 feature columns**.
- **Feature generation:** every model used the same core feature set: temporal features, route context, congestion proxies, weather intensity, frequency encodings, and train-only smoothed historical rates.
- **Leakage control:** the train-only encodings were fit on the training partition and then applied to validation or holdout data with smoothing and fallbacks for unseen groups.
- **Model-specific adaptation:** the histogram boosting models kept collapsing toward the majority class when the dense `_historical_` target-encoding columns stayed in the matrix. We removed those columns for `hist_gradient_boosting` and `hierarchical_hist_gradient_boosting`, and that fixed the issue.
- **Cross-validation:** inside training, we used stratified cross-validation to estimate generalization without touching the final testing set.
- **Hyperparameter tuning:** randomized CV search was implemented on best models.

11.4 Final Model-Selection Rule

Models were ranked by **balanced accuracy**, then tie-broken by **macro F1**, **average misclassification cost**, **cost reduction vs the on-time baseline**, and **weighted F1**. In practice, that meant we preferred models that still did useful work on the minority classes instead of just inflating overall accuracy.

More on this part in the results and evaluation section.

12 Experiment Tracking with MLflow

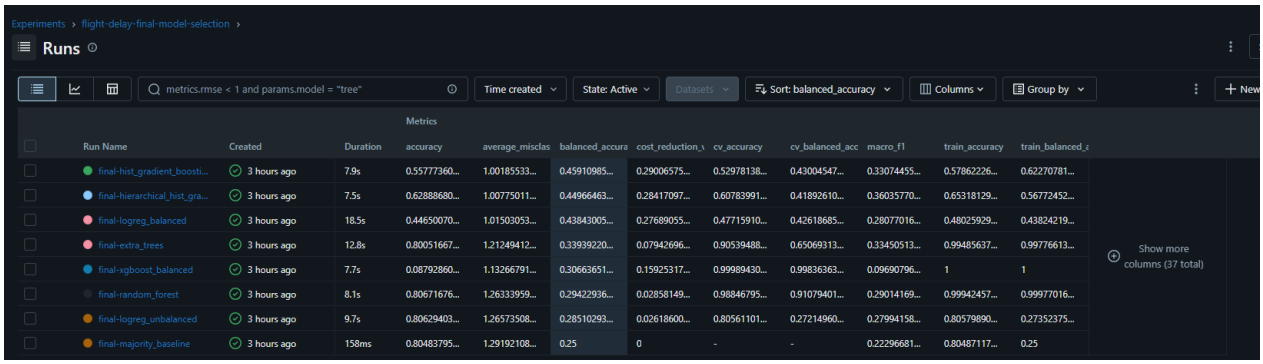
We tracked the full modeling workflow in **MLflow** the SQLite way (`mlflow.db`) and file-based artifacts under `mlartifacts/`.

12.1 What Each Run Logs

Each serious run stores the information required by the project brief:

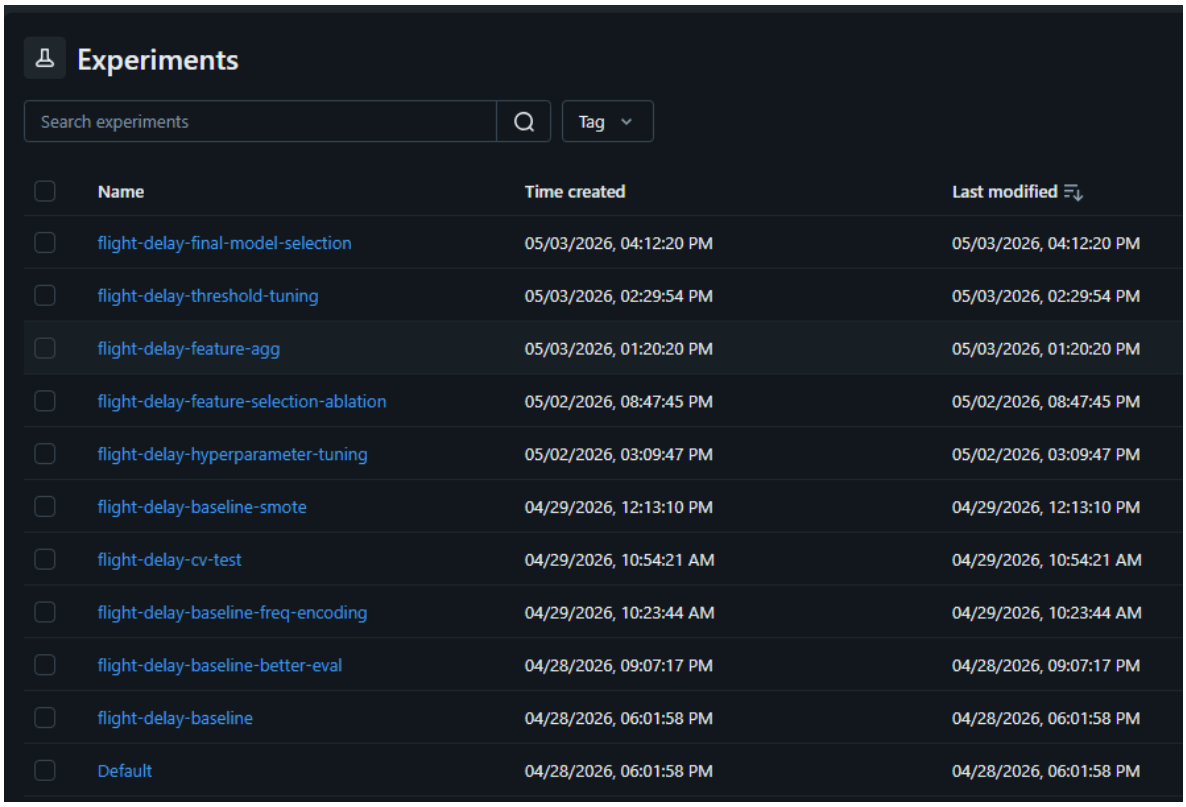
- **Model identity and setup** experiment name, run name, model family, algorithm, preprocessing choice, class-weight setting, optional SMOTE flag, and key hyperparameters.
- **Technical metrics** train, cross-validation, and/or holdout accuracy, balanced accuracy, macro F1, and weighted F1.
- **Business metrics** average misclassification cost and cost reduction relative to the `predict-all-on-time` operational baseline.
- **Artifacts** serialized model file when a fitted estimator exists, `evaluation_report.json`, and exported test predictions CSV for downstream error analysis.

12.2 Experiments



Run Name	Created	Duration	accuracy	average_misclas	balanced_accura	cost_reduction_\	cv_accuracy	cv_balanced_acc	macro_f1	train_accuracy	train_balanced_a
final-hist_gradient_boosti...	3 hours ago	7.9s	0.55777360...	1.00185533...	0.45910985...	0.29006575...	0.52978138...	0.43004547...	0.33074455...	0.57862226...	0.62270781...
final-hierarchical_hist_gra...	3 hours ago	7.5s	0.62888680...	1.00775011...	0.44966463...	0.28417097...	0.60783991...	0.41892610...	0.36035770...	0.65318129...	0.56772452...
final-logreg_balanced	3 hours ago	18.5s	0.44650070...	1.01503053...	0.43843005...	0.27689055...	0.47715910...	0.42618685...	0.28077016...	0.48025929...	0.43824219...
final-extra_trees	3 hours ago	12.8s	0.80051667...	1.21249412...	0.33939220...	0.07942696...	0.90539488...	0.65069313...	0.33450513...	0.99485637...	0.99776613...
final-xgboost_balanced	3 hours ago	7.7s	0.08792860...	1.13266791...	0.30663651...	0.15925317...	0.99989430...	0.99836363...	0.09690796...	1	1
final-random_forest	3 hours ago	8.1s	0.80671676...	1.26333959...	0.29422936...	0.02858149...	0.98846795...	0.91079401...	0.29014169...	0.99942457...	0.99977016...
final-logreg_unbalanced	3 hours ago	9.7s	0.80629403...	1.26573508...	0.28510293...	0.02618600...	0.80561101...	0.27214960...	0.27994158...	0.80579890...	0.27352375...
final-majority_baseline	3 hours ago	158ms	0.80483795...	1.29192108...	0.25	0	-	-	0.22296681...	0.80487117...	0.25

Figure 10: MLflow comparison page



Experiments

Search experiments | Tag

Name	Time created	Last modified
flight-delay-final-model-selection	05/03/2026, 04:12:20 PM	05/03/2026, 04:12:20 PM
flight-delay-threshold-tuning	05/03/2026, 02:29:54 PM	05/03/2026, 02:29:54 PM
flight-delay-feature-agg	05/03/2026, 01:20:20 PM	05/03/2026, 01:20:20 PM
flight-delay-feature-selection-ablation	05/02/2026, 08:47:45 PM	05/02/2026, 08:47:45 PM
flight-delay-hyperparameter-tuning	05/02/2026, 03:09:47 PM	05/02/2026, 03:09:47 PM
flight-delay-baseline-smote	04/29/2026, 12:13:10 PM	04/29/2026, 12:13:10 PM
flight-delay-cv-test	04/29/2026, 10:54:21 AM	04/29/2026, 10:54:21 AM
flight-delay-baseline-freq-encoding	04/29/2026, 10:23:44 AM	04/29/2026, 10:23:44 AM
flight-delay-baseline-better-eval	04/28/2026, 09:07:17 PM	04/28/2026, 09:07:17 PM
flight-delay-baseline	04/28/2026, 06:01:58 PM	04/28/2026, 06:01:58 PM
Default	04/28/2026, 06:01:58 PM	04/28/2026, 06:01:58 PM

Figure 11: MLflow experiment list

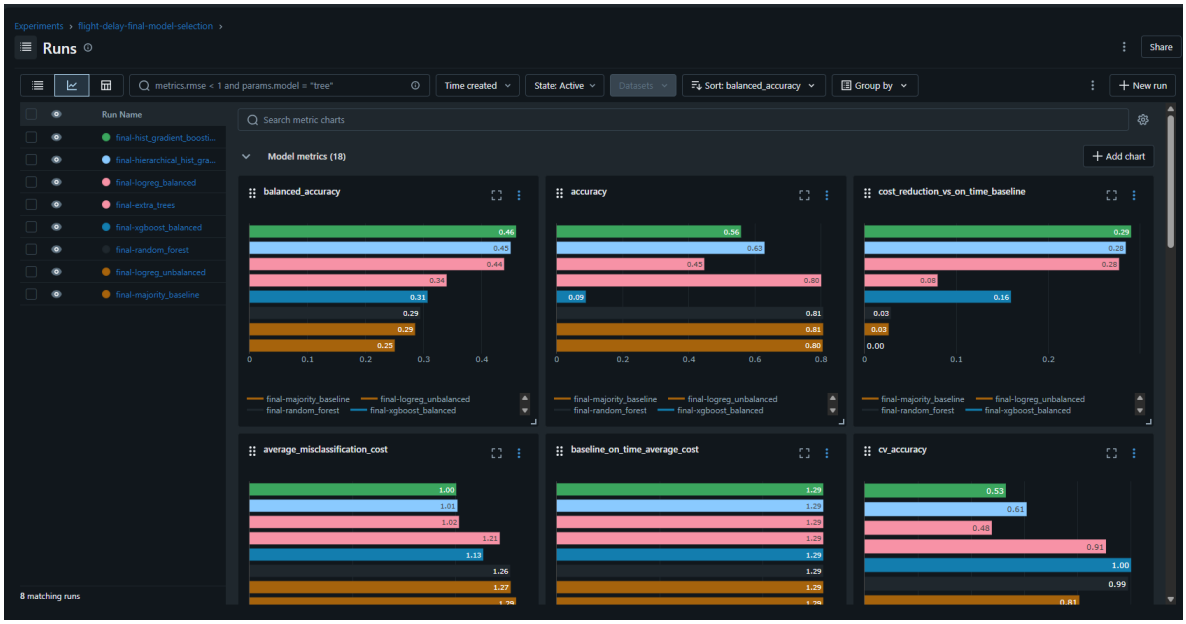


Figure 12: Chart view of the final MLflow runs

12.3 Why MLflow Matters

MLflow mattered because the modeling pipeline changed several times during the project. We tried different feature sets, imbalance strategies, model families, and evaluation rules. Without tracked runs, it would have been too easy to mistake a bad improvement for a real one, especially when some tuning results looked good in cross-validation and then weakened on test (and other silly mistakes that we faced).

13 Testing

The project brief asks for unit tests, integration tests, coverage, and a short discussion of issues found. This section follows that structure. The repository uses **pytest**, and the latest validation run finished with **30 passing tests** and **51% total line coverage** across the Python package.

13.1 Unit and Integration Test Coverage

Most tests are **unit-style checks** built on small synthetic data frames. That made it easier to isolate edge cases and check the exact rules we care about, such as where `on_time` becomes `minor_delay`, or whether historical-rate features are built from the train split without leaking target information.

The suite also includes **integration-style checks** around the modeling pipeline. The training tests run the full `train_and_log_model` path on temporary artifacts and check that it writes the expected model file, JSON evaluation output, predictions CSVs, tracking database, and MLflow artifacts.

14 Results & Evaluation

The final evaluation used technical metrics and business metrics side by side. We kept the usual classification scores, but we did not want the report to stop at them because this is an imbalanced problem and some mistakes matter much more than others. On the business side, we used a simple misclassification-cost scheme that makes missed disruption events more expensive than false alarms

14.1 Why These Metrics Were Used

What Each Metric Tells Us

Metric	Why We Used It	Business Meaning
Accuracy	We kept accuracy because it is the easiest overall number to read and compare.	-
Balanced accuracy	This was the **main ranking metric**. It averages recall across the four classes, so every class counts equally.	-
Macro F1	We used macro F1 to balance precision and recall while still giving equal weight to each class.	-
Weighted F1	We kept weighted F1 as a secondary summary because it follows the real class mix in the data.	-
Per-class precision, recall, and F1	We reported the class-wise scores for the best-model review and the error analysis.	-
Average misclassification cost	This was the **main business metric**. It assigns a penalty to each wrong prediction and averages that cost over all flights. Lower is better.	It turns model errors into something closer to operational impact. Missing a cancelled flight hurts much more than falsely warning on a normal flight.
Cost reduction vs on-time baseline	We compared every model with a naive rule that predicts every flight as `on_time`. Higher is better.	A positive value means the model saves expected business cost. A zero or negative value means the model is not helping enough to justify using it.

The business-cost weights were chosen to match what these mistakes mean in operations. A false alarm on an `on_time` flight has a small cost because it may trigger extra monitoring or passenger messages, but it usually does

not disrupt the day badly. Missing a `minor_delay` is more expensive because updates start late and connection handling gets weaker. Missing a `major_delay` is worse. Missing a `cancelled` flight has the highest cost because the airline loses precious time on the most urgent disruption response.

14.2 Final Model Comparison on the Locked Holdout Set

Model	Train Acc.	Train Bal. Acc.	Test Acc.	Test Bal. Acc.	Avg. Cost	Cost Reduction
HistGradientBoosting	0.579	0.623	0.558	**0.459**	**1.002**	**0.290**
Hierarchical HistGradientBoosting	0.630	0.515	0.624	0.449	1.010	0.282
Logistic Regression (balanced)	0.480	0.438	0.447	0.438	1.015	0.277
Extra Trees	0.995	0.998	0.801	0.339	1.212	0.079
XGBoost (balanced)	1.000	1.000	0.088	0.307	1.133	0.159
Random Forest	0.999	1.000	0.807	0.294	1.263	0.029
Logistic Regression (unbalanced)	0.806	0.274	0.806	0.285	1.266	0.026
Majority baseline	0.805	0.250	0.805	0.250	1.292	0.000

Three points stand out:

- **Accuracy alone is misleading.** The majority baseline, unbalanced logistic regression, random forest, and extra trees all exceed 0.80 plain accuracy because the dataset is dominated by `on_time` flights..
- **HistGradientBoosting won the final comparison.** It achieved the highest test balanced accuracy while giving the largest cost reduction.
- **The hierarchical model was competitive but not best under the primary objective.** It gave the highest macro F1 (**0.353**) and higher plain accuracy than the winner, but it still was worse than the single-stage HistGradientBoosting model on balanced accuracy, which was the locked selection criterion.

14.3 Best Model Detailed Performance

Best Final Run: `final-hist_gradient_boosting`

Field	Value
Test accuracy	0.558
Test balanced accuracy	**0.459**
Test macro F1	0.331
Test weighted F1	0.624
Average misclassification cost	**1.002**
Cost reduction vs on-time baseline	**0.290** absolute cost units
Train vs test balanced accuracy	0.623 vs 0.459 (slight overfitting, but still best generalization among final candidates)

Class	Precision	Recall	F1	Support
On Time	0.890	0.611	0.725	17,135
Minor Delay	0.162	0.325	0.217	2,141
Major Delay	0.149	0.295	0.198	1,670
Cancelled	0.108	**0.605**	0.183	344

It is useful when the goal is to catch severe events, especially cancellations, but it pays for that sensitivity with many false alarms on flights that were actually on time.

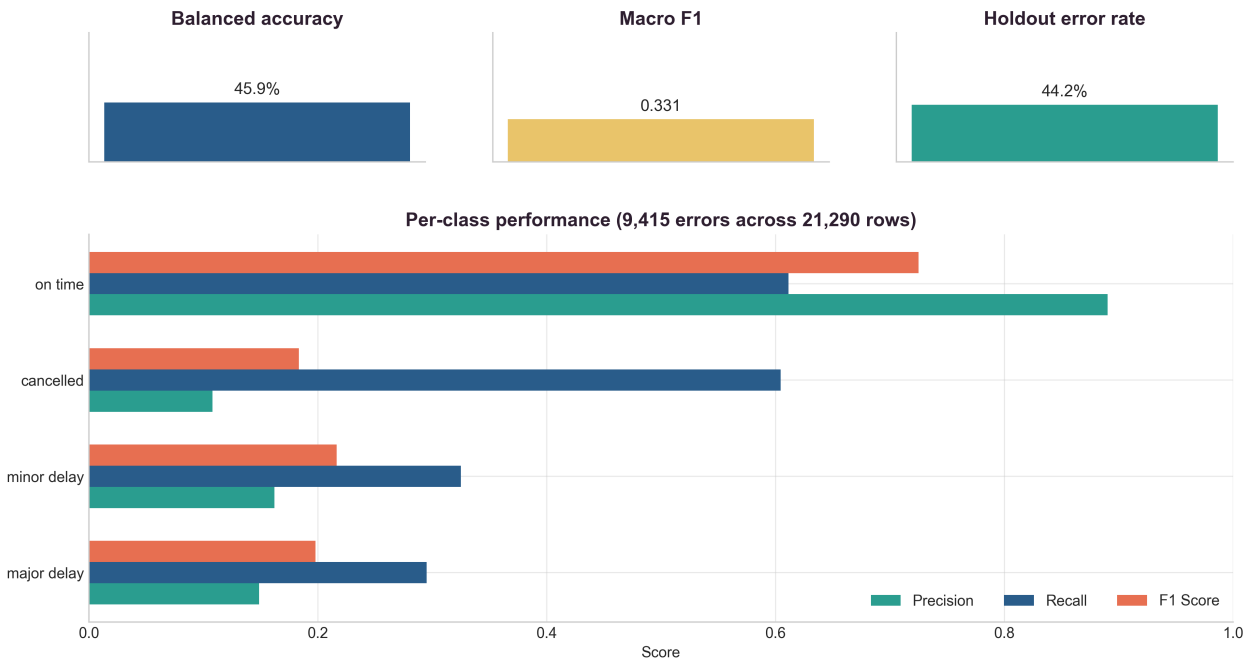


Figure 13: per-class precision, recall, and F1 for the four delay classes.

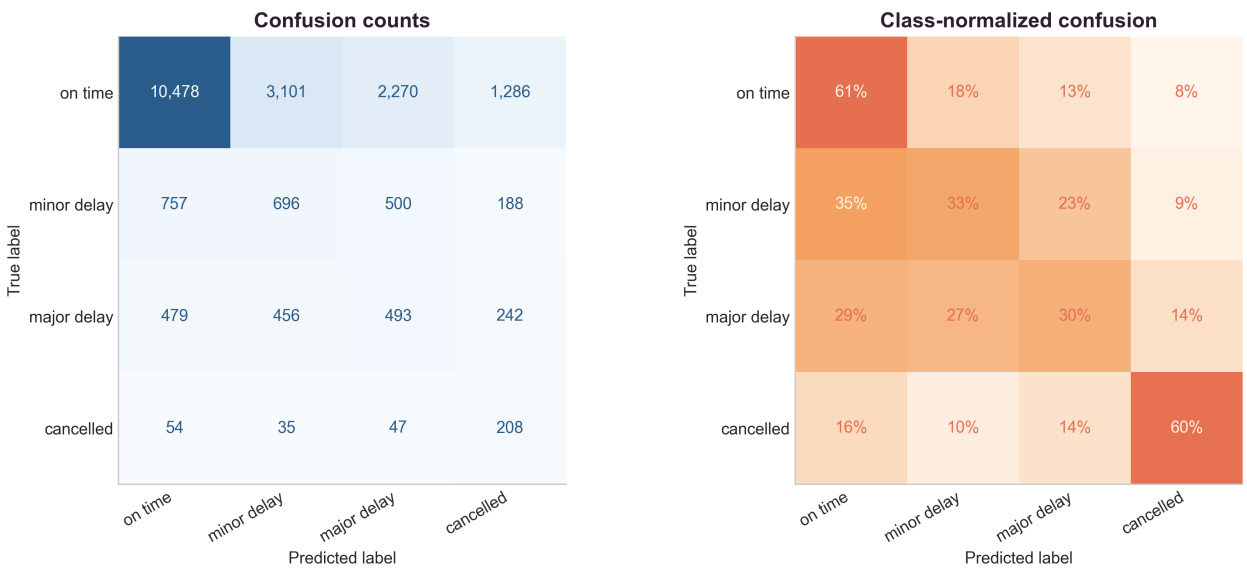


Figure 14: Confusion metrices

14.4 Error Analysis and Business Interpretation

Most of the mistakes came from **over-predicting disruption** rather than missing every disruption case entirely. The three most common confusion pairs were:

- `on_time` → *minordelay*
- `on_time` → *majordelay*
- `on_time` → *cancelled*

For the stakeholder, this means the model is **useful for pre-departure prioritization**. It catches a meaningful share of the major disruption cases and reduces cost relative to an `all-on-time` rule. Still, it should be treated as a **decision-support signal**, not as a fully automated trigger, because the false-positive rate on nominally on-time flights is still high.

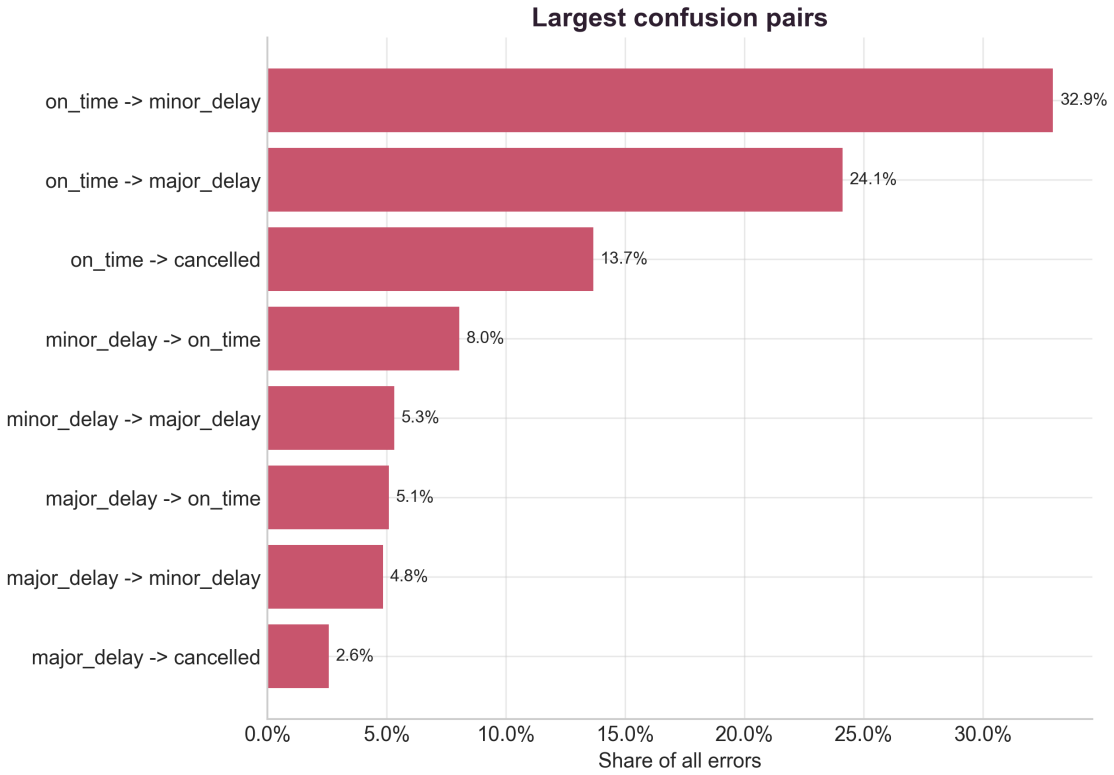


Figure 15: Largest confusion pair

15 Code Automation

The project was structured so that the main workflow can be reproduced from the command line without manually editing notebooks or changing paths inside the code. We kept the automation simple on purpose: each stage of the pipeline is exposed either as a **Make target** or as a **Python CLI entrypoint**.

15.1 Makefile-Based Pipeline Automation

The first automation layer is the project `Makefile`. It defines the common development commands used throughout the semester, including environment setup, linting, formatting, testing, and the main data-to-model pipeline. The project rules are chained in dependency order, so commands such as `make train` or `make evaluate` automatically execute the upstream steps first. In practice, that means the workflow moves in a consistent sequence:

`requirements` → `data` → `features` → `train` → `evaluate`

The same Makefile also includes `make lint`, `make format`, `make test`, and `make mlflow-ui`, so local quality checks and experiment inspection use the same interface as the modeling steps instead of relying on ad hoc terminal commands.

16 Continuous Integration

16.1 Current CI Workflow

- `poetry run make lint`
- `poetry run make test`

This means each pull request is checked for both **code quality** and **basic behavioral correctness**.

17 Limitations & Future Work

17.1 Current Limitations

- **Low mutual-information for pre-departure features**: we found that the notebooks that use this datasets have huge data leakage so they get high results but the classification task itself without the leaking columns is a hard task even when augmented with weather data.
- **The weather data is historical not forecast**
- **The data is scoped in one year only**
- **The missing tail_number (plane_identifier) reduced feature engineering** Tail number could have been used to track the cascading singal of a plane that has a delay in a certain day.

To conclude, the current system is already strong enough to support **prioritization and early warning**, but not strong enough for full automation in an airline.

18 Team Contributions

The distribution of work is detailed below:

Work Distribution

Team Member	Role	Key Responsibilities
Akram Hany	Data Engineer	Data acquisition, merging multiple sources, establishing thorough validation pipelines, Handled preprocessing.
Amir Anwar	ML Engineer	Engineered complex features, built and trained all 5+ classification models, tracked experiments, and conducted deep model evaluation.
Somia Saad	Data Analyst	Defined the core business problem, conducted rigorous Exploratory Data Analysis (EDA), interpreted model results for stakeholders, and maintained the overall quality of the report.
Ahmed Hamed	MLOps	Enforced clean code practices, wrote comprehensive unit and integration tests, set up project automation (Makefiles), configured CI pipelines.